

Universidade Federal da Fronteira Sul

Relatório Construção de um Analisador Sintático

Erickson Giesel Müller

2 de julho de 2026

1 Resumo

Este relatório descreve o desenvolvimento e a implementação da primeira etapa de um analisador sintático. O objetivo principal do trabalho consiste em validar a estrutura gramatical de um código-fonte a partir de uma Gramática Livre de Contexto (GLC). A metodologia adotada envolveu a construção de um algoritmo na linguagem Python baseado na abordagem de análise ascendente, integrado à fita de saída de um analisador léxico. O sistema implementado realiza a leitura dinâmica da gramática, aplica algoritmos de tratamento para a eliminação de símbolos inúteis e fatoração, e calcula os conjuntos matemáticos FIRST e FOLLOW. Para o motor de reconhecimento, utiliza-se uma tabela de parsing SLR consumida por um autômato com pilha. Os resultados demonstram que o analisador é capaz de processar os tokens, aceitando estruturas válidas da linguagem e emitindo diagnósticos precisos de erros sintáticos ao interceptar anomalias na fita, cumprindo todos os requisitos exigidos para a fase de análise sintática.

2 Introdução

No processo de construção de um compilador, a tradução de um código-fonte para uma linguagem de máquina ou intermediária é dividida em diversas fases lógicas. Enquanto a análise léxica atua como a primeira linha de processamento, agrupando caracteres em unidades com significado, chamados de tokens, é a análise sintática que assume a responsabilidade de validar se a disposição desses tokens obedece às regras estruturais da linguagem projetada.

Este relatório documenta o desenvolvimento da etapa de análise sintática de um compilador. O projeto tem como escopo a implementação de um analisador sintático, projetado para consumir a fita de saída do analisador léxico desenvolvido no trabalho 1. O analisador valida as cadeias de entrada contra uma Gramática Livre de Contexto utilizando uma tabela de parsing da classe SLR. O código foi desenvolvido na linguagem Python e abrange algoritmos como a eliminação de símbolos inúteis, fatoração de regras para remoção de ambiguidades e a construção dos conjuntos matemáticos FIRST e FOLLOW.

3 Referencial Teórico

A análise sintática é a segunda fase do processo de compilação, responsável por verificar se a estrutura dos tokens gerados pelo analisador léxico está de acordo com as regras

formais da linguagem. Para descrever a sintaxe de linguagens de programação, utilizam-se as Gramáticas Livres de Contexto (GLC).

Uma GLC é formalmente definida por um conjunto de variáveis (ou Não-Terminais), um conjunto de símbolos terminais (tokens), um conjunto de regras de produção e um símbolo inicial. No escopo deste projeto, a representação computacional da gramática é inicializada pela função `carregar_glc(self, caminho_arquivo)`. Este método lê o arquivo de texto contendo as regras de produção e o converte dinamicamente para a estrutura de dicionário `self.glc` na memória do programa, mapeando os Não-Terminais para as suas respectivas listas de derivações.

3.1 Tratamento da Gramática

— GLC Final Pós-Processamento —

```
<PROGRAMA> -> [['<DECLARACAO>'], ['<PROGRAMA>', '<DECLARACAO>']]  
<DECLARACAO> -> [['definir', 'variavel', 'como', 'tipo', 'string'], ['<tipo>', 'variavel',  
'=', 'new', '<classe>']]
```

Terminais detectados: '<tipo>', 'como', 'variavel', 'tipo', 'definir', 'new', 'string', '=', '<classe>'

Para que uma gramática seja adequadamente processada por um autômato (especialmente em abordagens tabulares), ela frequentemente precisa passar por transformações estruturais que garantam seu determinismo e validade.

O primeiro passo consiste na eliminação de símbolos inúteis, que são variáveis que nunca geram uma cadeia de terminais (não-geradores) ou que nunca podem ser alcançadas a partir do símbolo inicial (inacessíveis). O algoritmo implementado na função `eliminar_inuteis(self)` atua dividindo este processamento exatamente nessas duas fases: primeiramente, varre a gramática para encontrar e preservar apenas os símbolos geradores; em seguida, realiza uma travessia a partir do símbolo inicial para reter apenas os símbolos alcançáveis.

O segundo tratamento aplicado é a fatoração da GLC. A ambiguidade estrutural surge quando uma gramática possui regras de produção para um mesmo Não-Terminal que iniciam com o mesmo prefixo, exigindo que o analisador tente adivinhar qual caminho seguir. A função `fatorar_glc(self)` resolve este não-determinismo procurando prefixos em comum nas opções de derivação. Ao encontrar repetições, o algoritmo isola o prefixo e cria novos estados artificiais para derivar os sufixos restantes.

3.2 Conjuntos FIRST e FOLLOW

O núcleo matemático preditivo de um analisador sintático baseia-se nos conjuntos FIRST e FOLLOW. O conjunto $FIRST(\alpha)$ agrupa os símbolos terminais que iniciam as cadeias derivadas a partir da sequência α . Já o conjunto $FOLLOW(A)$ agrupa os terminais que podem aparecer imediatamente à direita do Não-Terminal A em alguma forma sentencial válida.

No sistema desenvolvido, esses conjuntos formam a base para o reconhecimento das transições de estados. Eles estão puramente implementados e são calculados de forma iterativa pelas funções `first(self, term)`, pela função auxiliar `first_seq(self, rhs, first_dict)` que é responsável por tratar a propagação da palavra vazia (ε) ao longo da cadeia e pela função `follow(self)`. Para garantir a eficiência de acesso durante

o consumo da fita pelo motor de reconhecimento, os resultados computados são salvos definitivamente nas variáveis de instância `self.first_sets` e `self.follow_sets`.

3.3 Tabela de Parsing SLR e Algoritmo de Mapeamento

Os analisadores sintáticos ascendentes (Bottom-Up) constroem a árvore de derivação a partir das folhas (tokens) em direção à raiz (símbolo inicial). Para gerenciar essa construção, utiliza-se um Autômato com Pilha guiado por uma Tabela de Parsing, sendo a classe SLR (Simple LR) uma das mais eficientes de se implementar.

A tabela SLR é dividida em duas partes: a função ACTION, que determina a ação a ser tomada com base no estado atual e no token lido; e a função GOTO, que dita o próximo estado após uma redução. As operações principais da máquina de estados são o *Shift* (empilhar o token e transitar para um novo estado) e o *Reduce* (substituir um corpo de regra pelo seu Não-Terminal correspondente, desempilhando os estados anteriores).

No projeto, a estrutura dessa matriz matemática foi carregada pela função `carregar_tabela_slr()`. Já a implementação do algoritmo de mapeamento que consome a fita de entrada encontra-se na função `executar_parser(self)`. Este algoritmo controla um laço de repetição acoplado a uma lista que simula a pilha (`pilha = [0]`), consultando iterativamente a tabela para aceitar a cadeia (`acc`) ou emitir um diagnóstico de erro sintático ao encontrar um cruzamento vazio.

SLR grammar (" is ϵ):

```

(0) PROGRAMA ->
(1) DECLARACAO
(2) PROGRAMA ->
DECLARACAO
DECLARACAO -> definir
variavel como tipo string
DECLARACAO -> TIPO_TOKEN
variavel = new CLASSE_TOKEN

```

FIRST / FOLLOW table

Nonterminal	FIRST	FOLLOW
PROGRAMA	{ ϵ , TIPO_TOKEN}	{ ϵ , TIPO_TOKEN}
DECLARACAO	{ ϵ , TIPO_TOKEN}	{ ϵ , TIPO_TOKEN}

SLR closure table

Goto	Kernel	State	Closure
goto(0, DECLARACAO)	(PROGRAMA -> DECLARACAO)	0	(PROGRAMA -> DECLARACAO; DECLARACAO -> definir variavel como tipo string; DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)
goto(0, definir)	(DECLARACAO -> definir variavel como tipo string)	1	(DECLARACAO -> definir variavel como tipo string)
goto(0, TIPO_TOKEN)	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)	2	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)
goto(2, variavel)	(DECLARACAO -> definir variavel como tipo string)	4	(DECLARACAO -> definir variavel como tipo string)
goto(2, como)	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)	5	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)
goto(5, =)	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)	7	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)
goto(5, TIPO_TOKEN)	(DECLARACAO -> definir variavel como tipo string)	8	(DECLARACAO -> definir variavel como tipo string)
goto(7, new)	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)	9	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)
goto(8, string)	(DECLARACAO -> definir variavel como tipo string)	10	(DECLARACAO -> definir variavel como tipo string)
goto(9, CLASSE_TOKEN)	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)	11	(DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN)

LR table

State	ACTION					GOTO
	definir	variavel	como	tipo	string	TIPO_TOKEN
0						
1	acc					
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						

Input (tokens): id * id * id

Maximum number of steps: 100

Trace

Step	Stack	Input	Action	Trace
1	0	id * id * id		

Fonte: SLR Parser Generator

3.4 Tabela de Símbolos

A Tabela de Símbolos é uma estrutura de dados fundamental que permeia todas as fases da compilação. Seu papel é armazenar informações sobre os identificadores e tokens encontrados no código-fonte, servindo como a principal via de comunicação entre o analisador léxico, o sintático e as futuras fases de análise semântica e geração de código.

No escopo desta implementação, a tabela é inicialmente construída e populada durante a fase léxica (armazenando linha, identificador, rótulo e palavra lida) e injetada na inicialização da classe `AnalisadorSintatico` através da variável `self.tabela_simbolos`. A arquitetura garante que, durante a operação de *Reduce* no mapeamento sintático, o compilador tenha o contexto estrutural exato para, nas etapas subsequentes, adicionar propriedades semânticas (como escopo e tipo de dado) a esta mesma tabela.

4 Implementação e Resultados

Considerando que o algoritmo usou os seguintes tokens e gramática em `fonte.txt`:

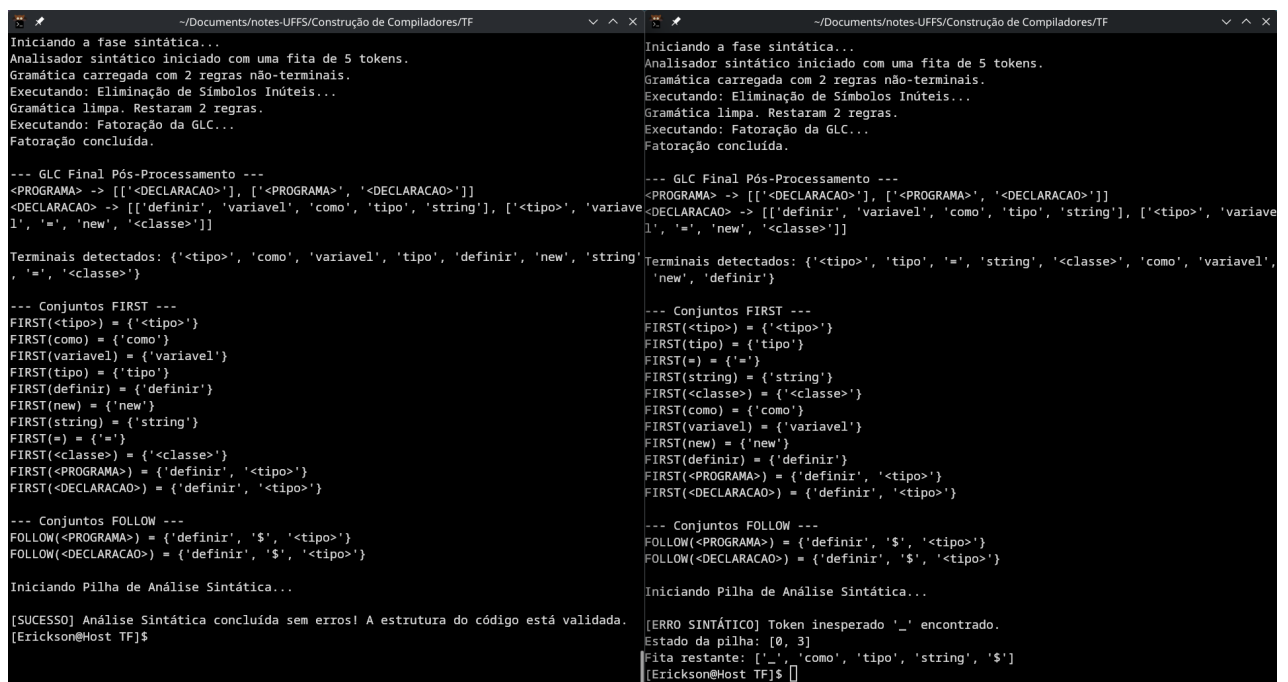
```
definir
como
tipo
string
```

```
S ::= a<A> | e<A> | i<A> | o<A> | u<A> | C<A> | T<A> | F<A> | G<A> | R<A>
A ::= f<A> | a<A> | e<A> | i<A> | {epsilon}
```

Realizaram-se os testes de sucesso e erro sintáticos com os seguintes códigos:

```
definir Teia como tipo string (sucesso)
definir Xeia como tipo string (erro)
```

Dos quais, o analisador respondeu, respectivamente, os seguintes outputs:



```
~/Documents/notes-UFFS/Construção de Compiladores/TF
Iniciando a fase sintática...
Analisador sintático iniciado com uma fita de 5 tokens.
Gramática carregada com 2 regras não-terminais.
Executando: Eliminação de Símbolos Inúteis...
Gramática limpa. Restaram 2 regras.
Executando: Fatoração da GLC...
Fatoração concluída.

--- GLC Final Pós-Processamento ---
<PROGRAMA> -> [['<DECLARACAO>'], ['<PROGRAMA>', '<DECLARACAO>']]
<DECLARACAO> -> [['definir', 'variavel', 'como', 'tipo', 'string'], ['<tipo>', 'variavel', '=', 'new', '<classe>']]

Terminais detectados: {'<tipo>', 'como', 'variavel', 'tipo', 'definir', 'new', 'string', '=', '<classe>' }

--- Conjuntos FIRST ---
FIRST(<tipo>) = {'<tipo>' }
FIRST(como) = {'como'}
FIRST(variavel) = {'variavel'}
FIRST(tipo) = {'tipo'}
FIRST(definir) = {'definir'}
FIRST(new) = {'new'}
FIRST(string) = {'string'}
FIRST(=) = {'='}
FIRST(<classe>) = {'<classe>' }
FIRST(<PROGRAMA>) = {'definir', '<tipo>' }
FIRST(<DECLARACAO>) = {'definir', '<tipo>' }

--- Conjuntos FOLLOW ---
FOLLOW(<PROGRAMA>) = {'definir', '$', '<tipo>' }
FOLLOW(<DECLARACAO>) = {'definir', '$', '<tipo>' }

Iniciando Pilha de Análise Sintática...
[SUCESSO] Análise Sintática concluída sem erros! A estrutura do código está validada.
[Erickson@Host TF]$

~/Documents/notes-UFFS/Construção de Compiladores/TF
Iniciando a fase sintática...
Analisador sintático iniciado com uma fita de 5 tokens.
Gramática carregada com 2 regras não-terminais.
Executando: Eliminação de Símbolos Inúteis...
Gramática limpa. Restaram 2 regras.
Executando: Fatoração da GLC...
Fatoração concluída.

--- GLC Final Pós-Processamento ---
<PROGRAMA> -> [['<DECLARACAO>'], ['<PROGRAMA>', '<DECLARACAO>']]
<DECLARACAO> -> [['definir', 'variavel', 'como', 'tipo', 'string'], ['<tipo>', 'variavel', '=', 'new', '<classe>']]

Terminais detectados: {'<tipo>', 'tipo', '=', 'string', '<classe>', 'como', 'variavel', 'new', 'definir'}

--- Conjuntos FIRST ---
FIRST(<tipo>) = {'<tipo>' }
FIRST(tipo) = {'tipo'}
FIRST(=) = {'='}
FIRST(string) = {'string'}
FIRST(<classe>) = {'<classe>' }
FIRST(como) = {'como'}
FIRST(variavel) = {'variavel'}
FIRST(new) = {'new'}
FIRST(definir) = {'definir'}
FIRST(<PROGRAMA>) = {'definir', '<tipo>' }
FIRST(<DECLARACAO>) = {'definir', '<tipo>' }

--- Conjuntos FOLLOW ---
FOLLOW(<PROGRAMA>) = {'definir', '$', '<tipo>' }
FOLLOW(<DECLARACAO>) = {'definir', '$', '<tipo>' }

Iniciando Pilha de Análise Sintática...
[ERRO SINTÁTICO] Token inesperado 'X' encontrado.
Estado da pilha: [0, 3]
Fita restante: ['_', 'como', 'tipo', 'string', '$']
[Erickson@Host TF]$
```

5 Conclusão

O desenvolvimento do analisador sintático demonstrou, na prática, a eficácia da integração entre as fases iniciais da construção de um compilador. A implementação rigorosa dos algoritmos de tratamento da Gramática Livre de Contexto (eliminação de símbolos inúteis e fatoração), aliada ao cálculo automatizado dos conjuntos matemáticos FIRST e FOLLOW, forneceu a base formal necessária para o funcionamento do motor de reconhecimento.

A escolha da abordagem ascendente (Bottom-Up) guiada por uma tabela de parsing SLR provou ser uma arquitetura eficiente. Como evidenciado nos testes práticos, o autômato de pilha operou de maneira previsível, consumindo tokens válidos até o estado de

aceitação, interceptando com precisão as anomalias estruturais e lexicais (bloqueando a execução e emitindo diagnósticos claros sobre o estado da máquina).

Por fim, a inicialização e o trânsito da Tabela de Símbolos durante o processo de análise sintática deixam a arquitetura do sistema perfeitamente engatilhada para as próximas fases. O compilador encontra-se estruturalmente sólido e pronto para receber a implementação da análise semântica.

Referências

- [1] SCHEFFEL, Roberto M. *Apostila Linguagens Formais e Autômatos*.
- [2] AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson Education do Brasil, 2007.
- [3] LOUDEN, K. C. **Compiladores: princípios e práticas**. São Paulo: Thomson Learning, 2004.